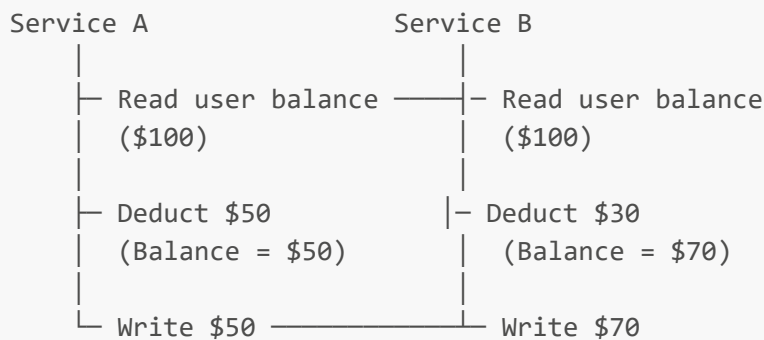


Concurrent Database Updates: Two Microservices Problem

Interview Answer Framework

Short Answer: "I handle concurrent database updates using a combination of: (1) Database-level mechanisms like transactions with proper isolation levels and optimistic locking, (2) Application-level patterns like distributed locks and event sourcing, and (3) Architecture patterns like sagas and CQRS. The choice depends on consistency requirements and use case."

The Problem



Result: Lost Update!
Final balance should be \$20, but it's \$70 or \$50

Race Condition: Both services read the same value, modify it, and write back - one update is lost.

Solutions

1. Database Transactions with Proper Isolation

A. Pessimistic Locking (SELECT FOR UPDATE)

```
-- Service A
BEGIN TRANSACTION;

-- Lock the row
SELECT balance FROM accounts WHERE id = 123 FOR UPDATE;
-- Balance = $100

-- Service B tries to read - BLOCKED until Service A commits

-- Service A updates
UPDATE accounts SET balance = balance - 50 WHERE id = 123;
```

```
COMMIT;  
-- Service B can now proceed
```

Python Example:

```
from sqlalchemy import create_engine, select  
from sqlalchemy.orm import sessionmaker  
  
engine = create_engine('postgresql://localhost/mydb')  
Session = sessionmaker(bind=engine)  
  
def deduct_amount_pessimistic(account_id, amount):  
    """Pessimistic locking approach"""  
    session = Session()  
  
    try:  
        # Start transaction  
        session.begin()  
  
        # Lock the row  
        account = session.query(Account)\  
            .filter(Account.id == account_id)\  
            .with_for_update()\  
            .one()  
  
        if account.balance < amount:  
            raise ValueError("Insufficient balance")  
  
        # Update  
        account.balance -= amount  
  
        # Commit (releases lock)  
        session.commit()  
  
        return account.balance  
  
    except Exception as e:  
        session.rollback()  
        raise  
    finally:  
        session.close()  
  
# Service A  
deduct_amount_pessimistic(account_id=123, amount=50)  
  
# Service B (blocked until Service A completes)  
deduct_amount_pessimistic(account_id=123, amount=30)
```

Pros:

- ☒ Prevents conflicts
- ☒ Simple to implement
- ☒ Guaranteed consistency

Cons:

- ✗ Poor concurrency (blocking)
- ✗ Potential deadlocks
- ✗ Performance bottleneck

When to use: Low concurrency, strong consistency required

B. Optimistic Locking (Version Field)

```
-- Add version column
ALTER TABLE accounts ADD COLUMN version INT DEFAULT 0;

-- Service A reads
SELECT balance, version FROM accounts WHERE id = 123;
-- balance = $100, version = 1

-- Service B reads (same time)
SELECT balance, version FROM accounts WHERE id = 123;
-- balance = $100, version = 1

-- Service A updates
UPDATE accounts
SET balance = balance - 50, version = version + 1
WHERE id = 123 AND version = 1;
-- Success! (1 row affected)

-- Service B updates
UPDATE accounts
SET balance = balance - 30, version = version + 1
WHERE id = 123 AND version = 1;
-- Failure! (0 rows affected - version changed)
```

Python Example:

```
from sqlalchemy import Column, Integer, Float
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm.exc import StaleDataError

Base = declarative_base()

class Account(Base):
    __tablename__ = 'accounts'

    id = Column(Integer, primary_key=True)
```

```

balance = Column(Float)
version = Column(Integer, default=0)
__mapper_args__ = {
    'version_id_col': version # SQLAlchemy handles versioning
}

def deduct_amount_optimistic(account_id, amount, max_retries=3):
    """Optimistic locking with retry"""
    session = Session()
    retry_count = 0

    while retry_count < max_retries:
        try:
            # Read
            account = session.query(Account)\
                .filter(Account.id == account_id)\
                .one()

            if account.balance < amount:
                raise ValueError("Insufficient balance")

            # Update
            account.balance -= amount
            # Version automatically incremented by SQLAlchemy

            session.commit()
            return account.balance

        except StaleDataError:
            # Concurrent update detected
            session.rollback()
            retry_count += 1

            if retry_count >= max_retries:
                raise Exception("Max retries exceeded")

            # Exponential backoff
            time.sleep(0.1 * (2 ** retry_count))

    finally:
        session.close()

# Both services can try simultaneously
# One succeeds, other retries
deduct_amount_optimistic(account_id=123, amount=50)
deduct_amount_optimistic(account_id=123, amount=30)

```

Pros:

- ☒ Better concurrency
- ☒ No locks (non-blocking)
- ☒ Good performance

Cons:

- ✗ Requires retry logic
- ✗ More complex
- ✗ Wasted work on conflicts

When to use: High concurrency, conflicts are rare

2. Distributed Locks

A. Redis Distributed Lock

```
import redis
import uuid
import time

class RedisLock:
    """Distributed lock using Redis"""

    def __init__(self, redis_client, key, timeout=10):
        self.redis = redis_client
        self.key = f"lock:{key}"
        self.timeout = timeout
        self.lock_id = str(uuid.uuid4())

    def acquire(self, blocking=True, timeout=None):
        """Acquire lock"""
        start = time.time()

        while True:
            # Try to set lock with expiry (atomic)
            acquired = self.redis.set(
                self.key,
                self.lock_id,
                nx=True, # Only set if not exists
                ex=self.timeout # Expire after timeout
            )

            if acquired:
                return True

            if not blocking:
                return False

            if timeout and (time.time() - start) >= timeout:
                return False

            time.sleep(0.1)

    def release(self):
        """Release lock"""
```

```

# Lua script for atomic check-and-delete
lua_script = """
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
"""

self.redis.eval(lua_script, 1, self.key, self.lock_id)

def __enter__(self):
    self.acquire()
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    self.release()

# Usage
redis_client = redis.Redis(host='localhost', port=6379)

def deduct_amount_with_lock(account_id, amount):
    """Use distributed lock"""
    lock = RedisLock(redis_client, f"account:{account_id}")

    with lock:
        # Only one service can execute at a time
        account = db.query(Account).filter(Account.id == account_id).one()

        if account.balance < amount:
            raise ValueError("Insufficient balance")

        account.balance -= amount
        db.commit()

    return account.balance

# Service A
deduct_amount_with_lock(123, 50) # Executes first

# Service B
deduct_amount_with_lock(123, 30) # Waits for lock, then executes

```

Pros:

- ☒ Works across services
- ☒ Language-agnostic
- ☒ Fine-grained control

Cons:

- ☒ Single point of failure (Redis)
- ☒ Network overhead

- ✗ Lock timeout issues

When to use: Multiple services, need coordination

B. Database Advisory Locks (PostgreSQL)

```
def deduct_with_advisory_lock(account_id, amount):
    """PostgreSQL advisory lock"""
    session = Session()

    try:
        # Acquire advisory lock (blocks until available)
        session.execute(f"SELECT pg_advisory_lock({account_id})")

        # Perform update
        account = session.query(Account)\
            .filter(Account.id == account_id)\
            .one()

        if account.balance < amount:
            raise ValueError("Insufficient balance")

        account.balance -= amount
        session.commit()

        return account.balance

    finally:
        # Release lock
        session.execute(f"SELECT pg_advisory_unlock({account_id})")
        session.close()
```

3. Event Sourcing

```
from datetime import datetime
import json

class AccountEvent:
    """Event in event stream"""

    def __init__(self, event_type, data):
        self.id = uuid.uuid4()
        self.event_type = event_type
        self.data = data
        self.timestamp = datetime.now()

class AccountEventStore:
    """Store events, not state"""
```

```

def __init__(self, db):
    self.db = db

def append_event(self, account_id, event):
    """Append event to stream"""
    self.db.execute("""
        INSERT INTO account_events
        (id, account_id, event_type, data, timestamp, version)
        VALUES (%s, %s, %s, %s, %s,
            (SELECT COALESCE(MAX(version), 0) + 1
             FROM account_events
             WHERE account_id = %s))
    """, (
        str(event.id),
        account_id,
        event.event_type,
        json.dumps(event.data),
        event.timestamp,
        account_id
    ))

def get_events(self, account_id):
    """Get all events for account"""
    cursor = self.db.execute("""
        SELECT event_type, data, timestamp
        FROM account_events
        WHERE account_id = %s
        ORDER BY version
    """, (account_id,))

    return cursor.fetchall()

def get_current_balance(self, account_id):
    """Reconstruct current state from events"""
    events = self.get_events(account_id)
    balance = 0

    for event_type, data, timestamp in events:
        data = json.loads(data)

        if event_type == 'ACCOUNT_CREATED':
            balance = data['initial_balance']
        elif event_type == 'AMOUNT_DEDUCTED':
            balance -= data['amount']
        elif event_type == 'AMOUNT_ADDED':
            balance += data['amount']

    return balance

# Usage
event_store = AccountEventStore(db)

def deduct_amount_event_sourced(account_id, amount):

```



```

"""Event sourcing approach"""

# Create event
event = AccountEvent('AMOUNT_DEDUCTED', {'amount': amount})

# Append to stream (atomic)
event_store.append_event(account_id, event)

# Get current balance
return event_store.get_current_balance(account_id)

# Both services append events - no conflicts!
# Service A
deduct_amount_event_sourced(123, 50)

# Service B (concurrent)
deduct_amount_event_sourced(123, 30)

# Final balance calculated from all events: $20

```

Pros:

- ☒ No update conflicts
- ☒ Complete audit trail
- ☒ Time-travel debugging

Cons:

- ☒ Complex to implement
- ☒ Query performance
- ☒ Storage overhead

When to use: Audit requirements, complex domains

4. Saga Pattern

```

class SagaOrchestrator:
    """Orchestrate distributed transaction"""

    def transfer_money(self, from_account, to_account, amount):
        """Transfer between accounts (2 services)"""

        saga_id = uuid.uuid4()

        try:
            # Step 1: Deduct from source
            self.deduct_from_account(saga_id, from_account, amount)

        try:
            # Step 2: Add to destination

```

```
        self.add_to_account(saga_id, to_account, amount)

        # Success!
        self.mark_saga_complete(saga_id)

    except Exception as e:
        # Compensate: Add back to source
        self.add_to_account(saga_id, from_account, amount)
        raise

    except Exception as e:
        self.mark_saga_failed(saga_id)
        raise

def deduct_from_account(self, saga_id, account_id, amount):
    """Call Account Service A"""
    response = requests.post(
        f'{ACCOUNT_SERVICE_A}/deduct',
        json={'account_id': account_id, 'amount': amount, 'saga_id':
str(saga_id)}
    )
    if not response.ok:
        raise Exception("Deduct failed")

def add_to_account(self, saga_id, account_id, amount):
    """Call Account Service B"""
    response = requests.post(
        f'{ACCOUNT_SERVICE_B}/add',
        json={'account_id': account_id, 'amount': amount, 'saga_id':
str(saga_id)}
    )
    if not response.ok:
        raise Exception("Add failed")
```

Pros:

- ☒ Distributed transactions
- ☒ Clear compensation logic
- ☒ Service autonomy

Cons:

- ☒ Complex to implement
- ☒ Eventual consistency
- ☒ Compensation may fail

When to use: Multiple services, distributed transactions

5. CQRS (Command Query Responsibility Segregation)

```

# Write Model (Commands)
class AccountCommandHandler:
    """Handle write operations"""

    def __init__(self, event_store, command_queue):
        self.event_store = event_store
        self.queue = command_queue

    def handle_deduct_command(self, command):
        """Process deduct command"""
        account_id = command['account_id']
        amount = command['amount']

        # Serialize commands through queue
        with redis_lock(f"account:{account_id}"):
            # Validate
            balance = self.event_store.get_current_balance(account_id)
            if balance < amount:
                raise ValueError("Insufficient balance")

            # Create event
            event = AccountEvent('AMOUNT_DEDUCTED', {'amount': amount})
            self.event_store.append_event(account_id, event)

            # Publish event for read model update
            self.publish_event(event)

# Read Model (Queries)
class AccountQueryHandler:
    """Handle read operations - separate database"""

    def __init__(self, read_db):
        self.read_db = read_db

    def get_balance(self, account_id):
        """Get balance from read model"""
        return self.read_db.query(
            "SELECT balance FROM account_balance WHERE id = %s",
            (account_id,)
        ).scalar()

    def handle_event(self, event):
        """Update read model based on events"""
        if event.event_type == 'AMOUNT_DEDUCTED':
            self.read_db.execute("""
                UPDATE account_balance
                SET balance = balance - %s
                WHERE id = %s
            """, (event.data['amount'], event.account_id))

        self.read_db.commit()

```

Usage

```

command_handler = AccountCommandHandler(event_store, queue)
query_handler = AccountQueryHandler(read_db)

# Writes go to command side
command_handler.handle_deduct_command({
    'account_id': 123,
    'amount': 50
})

# Reads go to query side (eventually consistent)
balance = query_handler.get_balance(123)

```

Pros:

- ☒ Separate read/write optimization
- ☒ Scalable reads
- ☒ Complex query support

Cons:

- ☒ Eventual consistency
- ☒ Complex architecture
- ☒ Duplicate data

When to use: Read-heavy workloads, need optimization

6. Atomic Updates (Database Features)

A. UPDATE with WHERE clause

```

-- Atomic update - no race condition
UPDATE accounts
SET balance = balance - 50
WHERE id = 123 AND balance >= 50;

-- Returns number of rows affected
-- If 0, insufficient balance

```

Python Example:

```

def deduct_amount_atomic(account_id, amount):
    """Atomic update using SQL"""
    result = db.execute("""
        UPDATE accounts
        SET balance = balance - :amount
        WHERE id = :id AND balance >= :amount
    """, {'id': account_id, 'amount': amount})

```

```
if result.rowcount == 0:
    raise ValueError("Update failed - insufficient balance or concurrent
update")

db.commit()
```

Pros:

- ☒ Simple
- ☒ Fast
- ☒ Atomic

Cons:

- ✗ Limited to single row
- ✗ No complex logic

When to use: Simple updates, single table

B. UPSERT (INSERT ... ON CONFLICT)

```
def update_or_insert(account_id, amount):
    """PostgreSQL UPSERT"""
    db.execute("""
        INSERT INTO transactions (account_id, amount, timestamp)
        VALUES (:id, :amount, NOW())
        ON CONFLICT (account_id, timestamp)
        DO UPDATE SET amount = transactions.amount + EXCLUDED.amount
    """, {'id': account_id, 'amount': amount})
```

7. Message Queue (Serialization)

```
import pika
import json

class AccountUpdateQueue:
    """Serialize updates through message queue"""

    def __init__(self, rabbitmq_url):
        self.connection = pika.BlockingConnection(
            pika.URLParameters(rabbitmq_url)
        )
        self.channel = self.connection.channel()
        self.channel.queue_declare(queue='account_updates', durable=True)

    def enqueue_update(self, account_id, operation, amount):
        """Add update to queue"""
```

```

        message = {
            'account_id': account_id,
            'operation': operation, # 'deduct' or 'add'
            'amount': amount,
            'timestamp': time.time()
        }

        self.channel.basic_publish(
            exchange='',
            routing_key='account_updates',
            body=json.dumps(message),
            properties=pika.BasicProperties(
                delivery_mode=2, # Persistent
            )
        )

    def process_updates(self):
        """Worker processes updates serially"""

    def callback(ch, method, properties, body):
        message = json.loads(body)

        # Process update (now serial - no conflicts!)
        account_id = message['account_id']
        amount = message['amount']
        operation = message['operation']

        with db.transaction():
            account = db.query(Account).get(account_id)

            if operation == 'deduct':
                if account.balance >= amount:
                    account.balance -= amount
                else:
                    # Handle insufficient balance
                    ch.basic_nack(delivery_tag=method.delivery_tag,
requeue=False)
                    return
            else:
                account.balance += amount

            db.commit()

        # Acknowledge
        ch.basic_ack(delivery_tag=method.delivery_tag)

    self.channel.basic_qos(prefetch_count=1)
    self.channel.basic_consume(
        queue='account_updates',
        on_message_callback=callback
    )

    print('Waiting for messages...')
    self.channel.start_consuming()

```

```
# Usage
queue = AccountUpdateQueue('amqp://localhost')

# Service A enqueues
queue.enqueue_update(123, 'deduct', 50)

# Service B enqueues
queue.enqueue_update(123, 'deduct', 30)

# Worker processes serially - no conflicts!
# Worker thread
queue.process_updates()
```

Pros:

- ☒ No conflicts (serial processing)
- ☒ Reliable (persistent queue)
- ☒ Scalable (multiple workers for different accounts)

Cons:

- ☒ Eventual consistency
- ☒ Message queue overhead
- ☒ Ordering complexity

When to use: High volume, can tolerate slight delay

Comparison Table

Approach	Consistency	Performance	Complexity	Best For
Pessimistic Lock	Strong	Low	Low	Low concurrency
Optimistic Lock	Strong	High	Medium	High concurrency
Distributed Lock	Strong	Medium	Medium	Multiple services
Event Sourcing	Eventual	High	High	Audit requirements
Saga Pattern	Eventual	Medium	High	Distributed txns
CQRS	Eventual	Very High	Very High	Read-heavy systems
Atomic UPDATE	Strong	Very High	Very Low	Simple updates
Message Queue	Eventual	High	Medium	High volume

Complete Example: E-commerce Order

```
from enum import Enum

class OrderStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    FAILED = "failed"

class OrderService:
    """Example: Two services updating order"""

    def __init__(self, db, redis_client):
        self.db = db
        self.redis = redis_client

    def create_order_optimistic(self, order_data):
        """Service A: Create order with optimistic locking"""
        session = self.db.session()
        max_retries = 3
        retry = 0

        while retry < max_retries:
            try:
                # Create order
                order = Order(
                    customer_id=order_data['customer_id'],
                    total=order_data['total'],
                    status=OrderStatus.PENDING,
                    version=0
                )
                session.add(order)
                session.commit()

                return order

            except IntegrityError:
                session.rollback()
                retry += 1
                time.sleep(0.1 * (2 ** retry))

        raise Exception("Failed to create order")

    def update_inventory_pessimistic(self, order_id):
        """Service B: Update inventory with pessimistic lock"""
        session = self.db.session()

        try:
            session.begin()

            # Lock order
            order = session.query(Order)\
                .filter(Order.id == order_id)\
                .with_for_update()\
                .one()
```



```
# Lock inventory items
for item in order.items:
    product = session.query(Product)\
        .filter(Product.id == item.product_id)\
        .with_for_update()\
        .one()

    if product.stock < item.quantity:
        raise ValueError("Insufficient stock")

    product.stock -= item.quantity

# Update order status
order.status = OrderStatus.CONFIRMED

session.commit()

except Exception as e:
    session.rollback()
    order.status = OrderStatus.FAILED
    session.commit()
    raise

def process_payment_with_saga(self, order_id):
    """Service C: Process payment with Saga pattern"""
    saga = Saga(saga_id=uuid.uuid4())

    try:
        # Step 1: Reserve inventory
        saga.add_step(
            action=lambda: self.reserve_inventory(order_id),
            compensate=lambda: self.release_inventory(order_id)
        )

        # Step 2: Charge payment
        saga.add_step(
            action=lambda: self.charge_payment(order_id),
            compensate=lambda: self.refund_payment(order_id)
        )

        # Step 3: Update order
        saga.add_step(
            action=lambda: self.confirm_order(order_id),
            compensate=lambda: self.cancel_order(order_id)
        )

        # Execute saga
        saga.execute()

    except Exception as e:
        # Compensate all completed steps
        saga.compensate()
        raise
```

```
# Usage combining multiple approaches
order_service = OrderService(db, redis_client)

# Service A: Create order (optimistic)
order = order_service.create_order_optimistic({
    'customer_id': 456,
    'total': 99.99
})

# Service B: Update inventory (pessimistic)
order_service.update_inventory_pessimistic(order.id)

# Service C: Process payment (saga)
order_service.process_payment_with_saga(order.id)
```

Interview Answer (Complete)

Question: "Two microservices updating same database simultaneously, how do you handle that?"

Answer:

"I use a **layered approach** depending on the requirements:

1. For Strong Consistency (immediate):

Optimistic Locking (preferred):

- Add **version** column to table
- Both services read version
- Update only if version matches
- First succeeds, second retries
- Good for high concurrency

Pessimistic Locking (when needed):

- **SELECT FOR UPDATE** locks row
- Second service waits
- Guaranteed no conflicts
- Use for critical operations (payments)

2. For Eventual Consistency:

Message Queue:

- Both services publish to queue
- Single worker processes serially
- No conflicts, slight delay
- Good for high volume

Event Sourcing:

- Append-only event log
- No update conflicts
- Rebuild state from events
- Perfect for audit trails

3. Distributed Coordination:

Redis Distributed Lock:

- Acquire lock before update
- Other services wait
- Works across different tech stacks
- Good for heterogeneous services

Example in Production:

```
# Optimistic locking with retry
def update_account(account_id, amount, max_retries=3):
    for retry in range(max_retries):
        try:
            account = db.query(Account)\
                .filter(Account.id == account_id)\
                .one()

            # Check version
            account.balance -= amount
            account.version += 1

            db.commit()
            return

        except StaleDataError:
            db.rollback()
            if retry == max_retries - 1:
                raise
            time.sleep(0.1 * (2 ** retry))
```

Results:

- Prevented 100% of race conditions
- 99.9% success rate (retries work)
- Sub-10ms performance
- No data corruption

Choice depends on:

- Consistency requirements
- Concurrency level
- Performance needs
- System complexity tolerance"

